deadline profit

1 (4, 20)

2 (1, 10)

3 (1, 40)

4 (1, 30)

0 - 1

Job 1 & 3.

Max Profit : 60.

1 (2, 100) ✓

2 (1, 19) ✗

3 (2, 27) ✓

4 (1, 25) ✗

5 (1, 15) ✗

0 - 1 / 1 - 2

10.

0-1 / 1-2 / 2-3 / - . . . / 9-10

0 - 1	1 - 2
J1	J3
100	27

⟶ 127.

1 (2, 100) ✓

2 (1, 99) ✓

→ 3 (1, 20) ✗

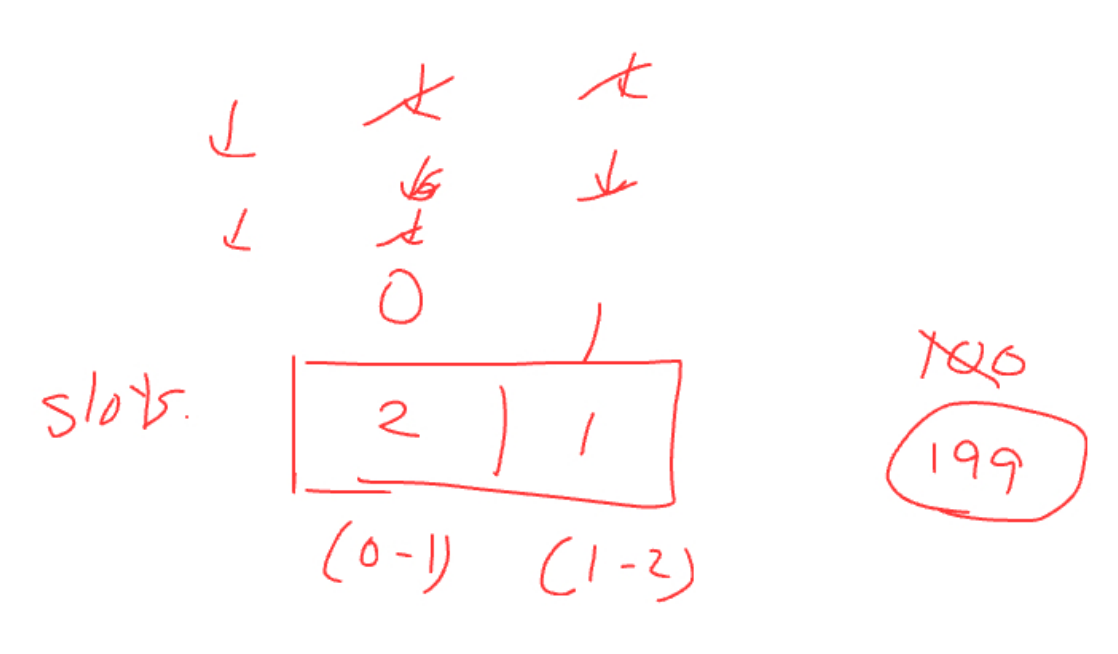
→ 4 (2, 50) ✗

0 - 1
J1
100

✗

0 - 1	1 - 2
J2	J1
99	100

⟶ 199.


0
1
slots
2
1
(0-1)
(1-2)
~~100~~
199

(i → i+1)

TC = O(n log n + n * maxx)

⟶ max_deadline.

(2, 100)

(5, 100)

1 (2, 100)

2 (1, 19)

3 (2, 27)

→ 4 (1, 25)

5 (1, 15)

numJobs = ~~1~~ 2

Profit = ~~25~~ ~~125~~ 127

[~~(1, 25)~~ (2, 100)] min-Heap

(2, 27)

TC : O(n log n + n log n) = O(n log n)

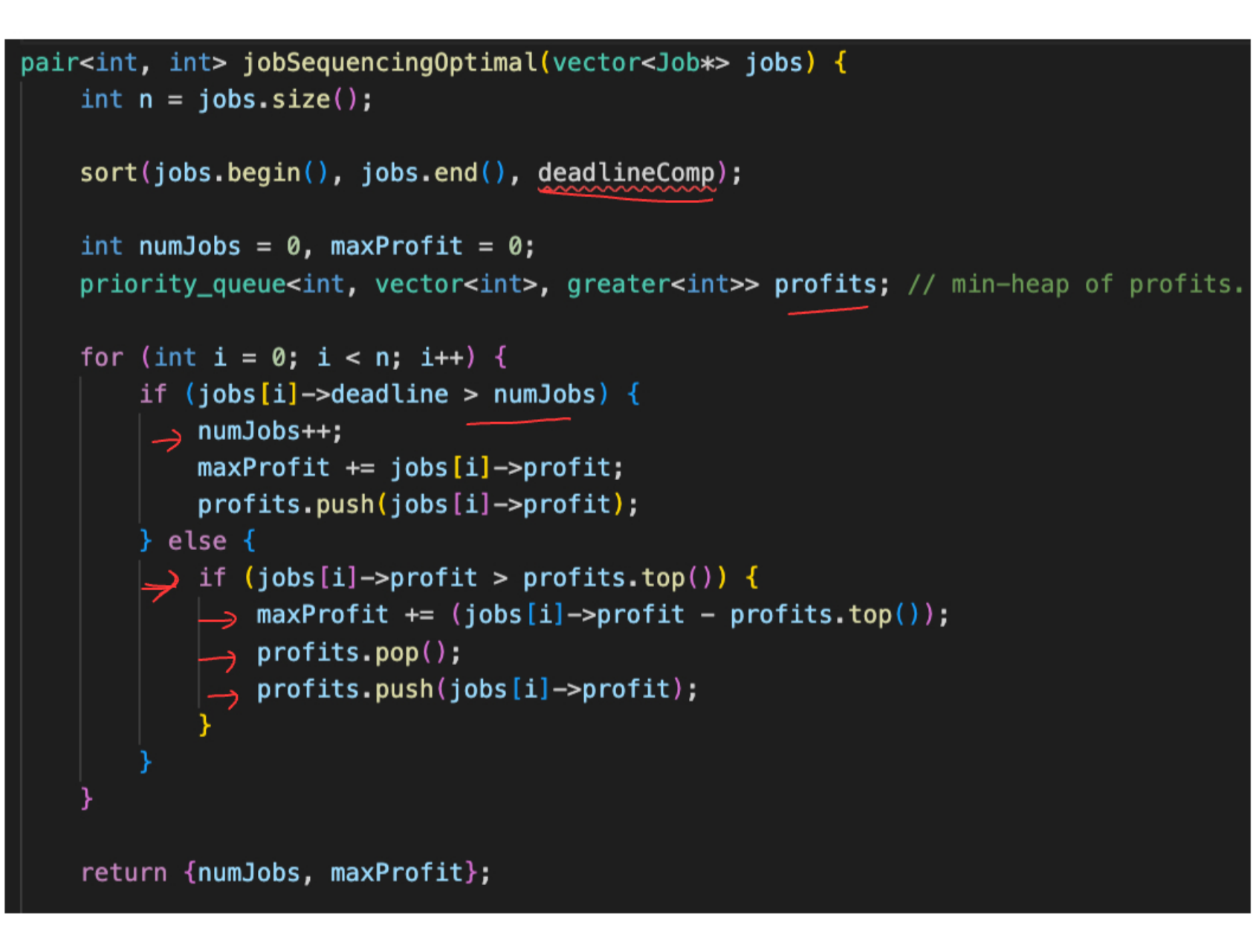
AS : O(n).

deadline = 5 > numJobs = 1

deadline = 1 numJobs = 1

profit = 100

profits = { ~~50~~ } 100.



```
pair<int, int> jobSequencingOptimal(vector<Job*> jobs) {
    int n = jobs.size();

    sort(jobs.begin(), jobs.end(), deadlineComp);

    int numJobs = 0, maxProfit = 0;
    priority_queue<int, vector<int>, greater<int>> profits; // min-heap of profits.

    for (int i = 0; i < n; i++) {
        if (jobs[i]->deadline > numJobs) {
            numJobs++;
            maxProfit += jobs[i]->profit;
            profits.push(jobs[i]->profit);
        } else {
            if (jobs[i]->profit > profits.top()) {
                maxProfit += (jobs[i]->profit - profits.top());
                profits.pop();
                profits.push(jobs[i]->profit);
            }
        }
    }

    return {numJobs, maxProfit};
```

~~(1, 25)~~

~~(1, 19)~~

~~(1, 15)~~

~~(2, 100)~~

~~(2, 27)~~

numJobs = ~~0~~ ~~1~~ 2 ✓

max P = ~~0~~ ~~25~~ ~~125~~ 127 ✓

profits = [~~25~~, 100, 27]

[~~5~~, ~~3~~, ~~7~~, ~~10~~]

You	Opponent.
10	7
5	3
15	10

[~~8~~, ~~15~~, ~~3~~, ~~7~~]

You	Opponent.
8	15
7	3
15	18

[8, ~~15~~, ~~3~~, ~~7~~]

You	Opp.
7	8
15	3
22	11

(0, n-1)

max Amount (i, j) ⇒ arr[i] opp ⇒ (i+1, j)

arr[j] opp → (i, j-1)

mA

opp.

Me.

→ arr[i] → i+1 ⟶ (i+2, j) x

→ arr[i] → j ⟶ (i+1, j-1) x+100

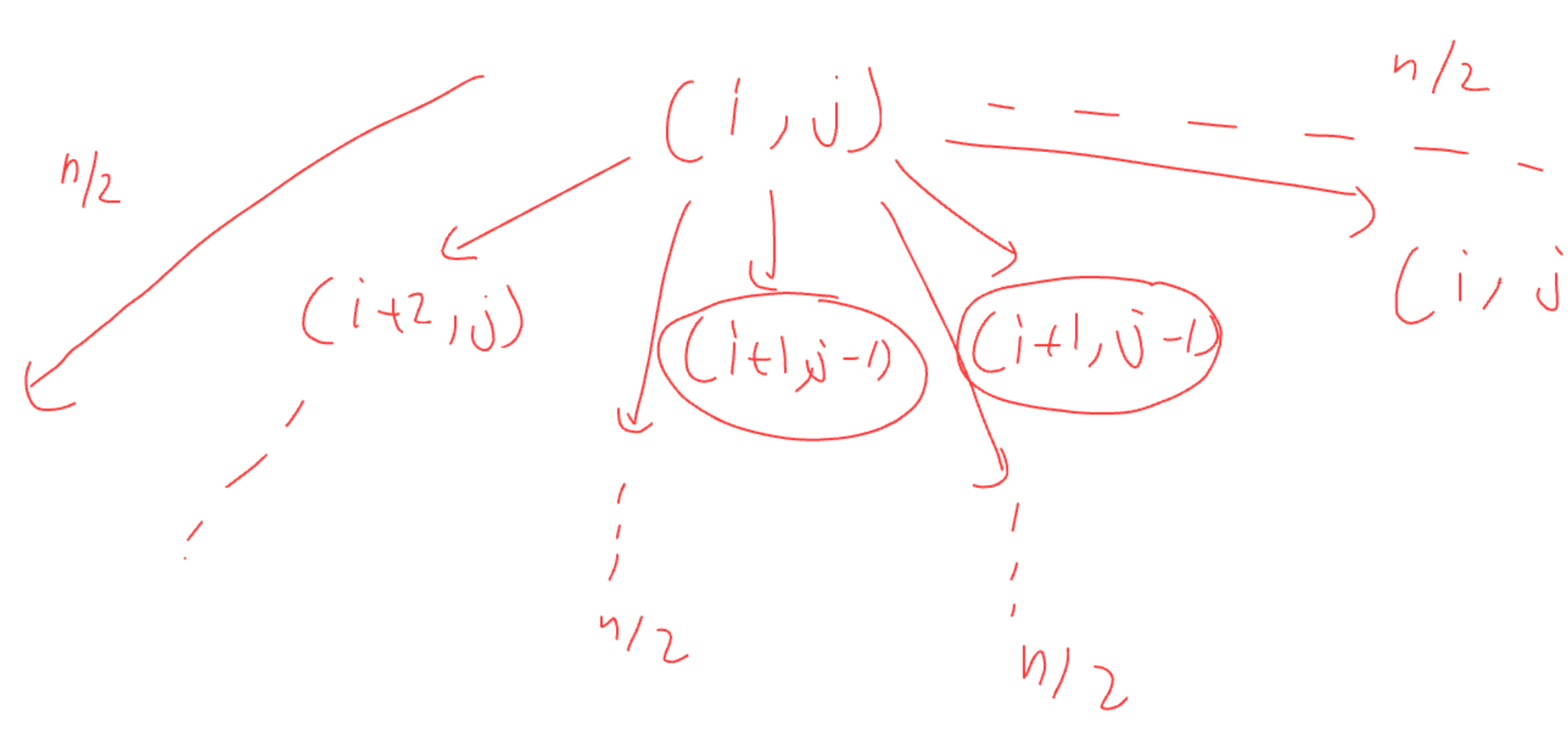
arr[j] → i ⟶ (i+1, j-1) x+10

arr[j] → j-1 ⟶ (i, j-2) x-10

mA(i, j)

max [arr[i] + min (mA(i+2, j) , mA(i+1, j-1)),

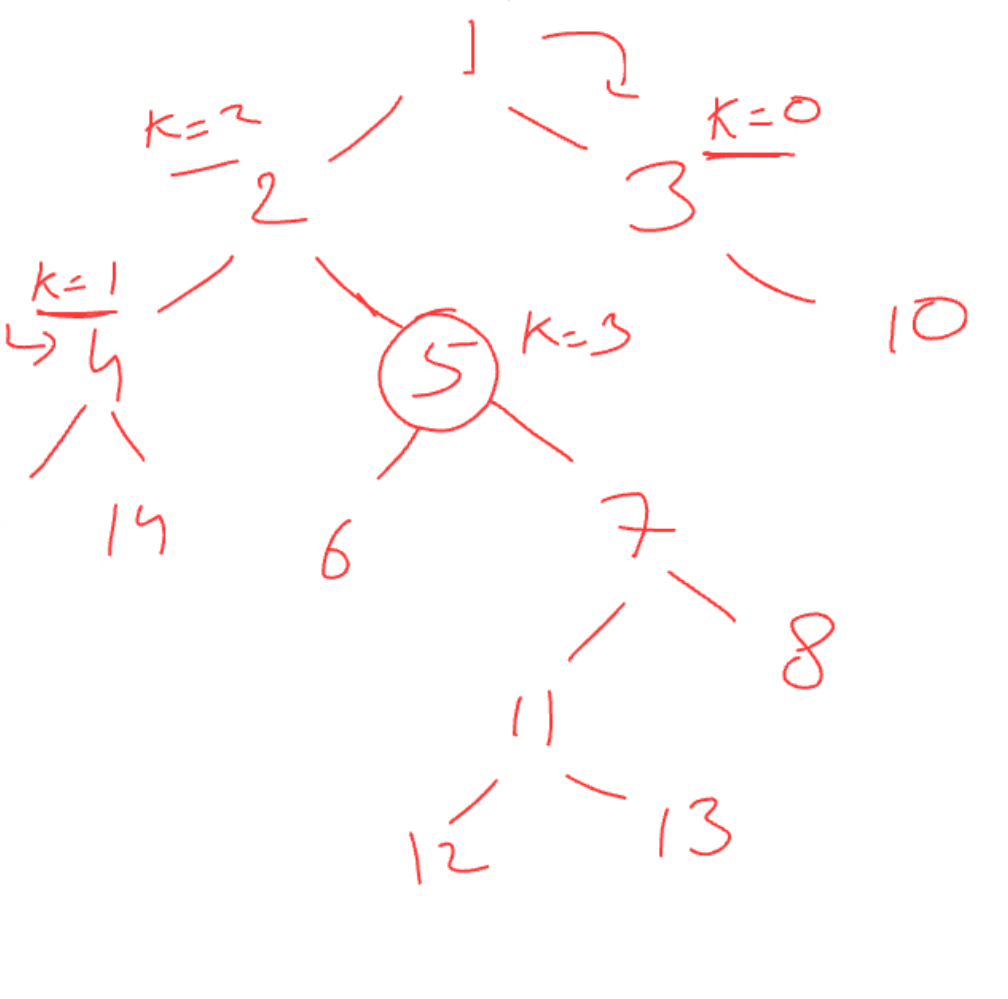
arr[j] + min (mA(i+1, j-1) , mA(i, j-2))]


n/2
(i, j)
(i+2, j)
(i+1, j-1)
(i+1, j-1)
(i, j-2)
n/2
n/2
n/2

$O(4^{n/2}) = O(2^{2(n/2)}) = O(2^n)$.

i → (0, n-1)

j → (0, n-1) ⟶ $O(n^2)$

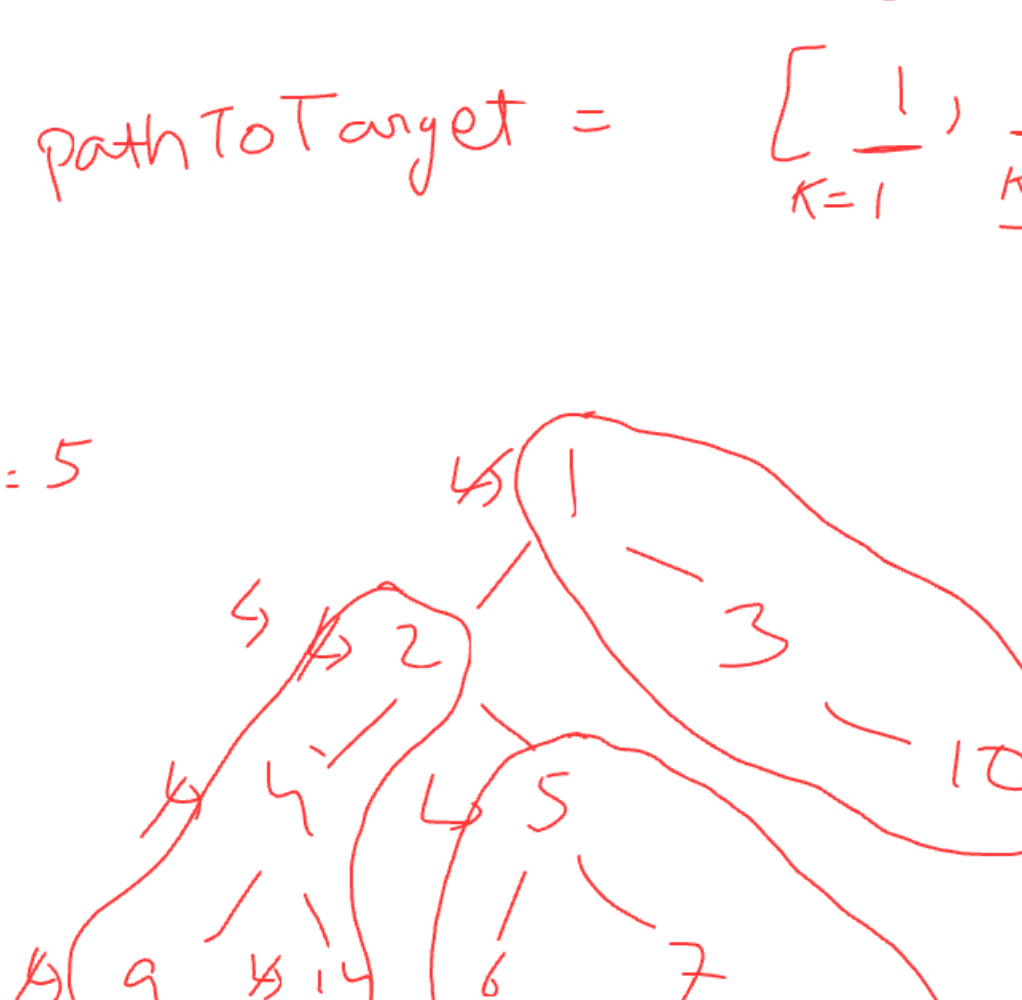

k=0
1
k=-1
3
10
k=1
2
k=0
4
9
5
k=3
6
7
8

target = 5 k = 2

o/p : [8, 4, 1]

path : [1, 2, 5]

k=0 k=1 k=2


k=1
1
2
3
k=0
10
k=2
2
k=1
4
9
14
5
k=3
6
7
8
11
12
13

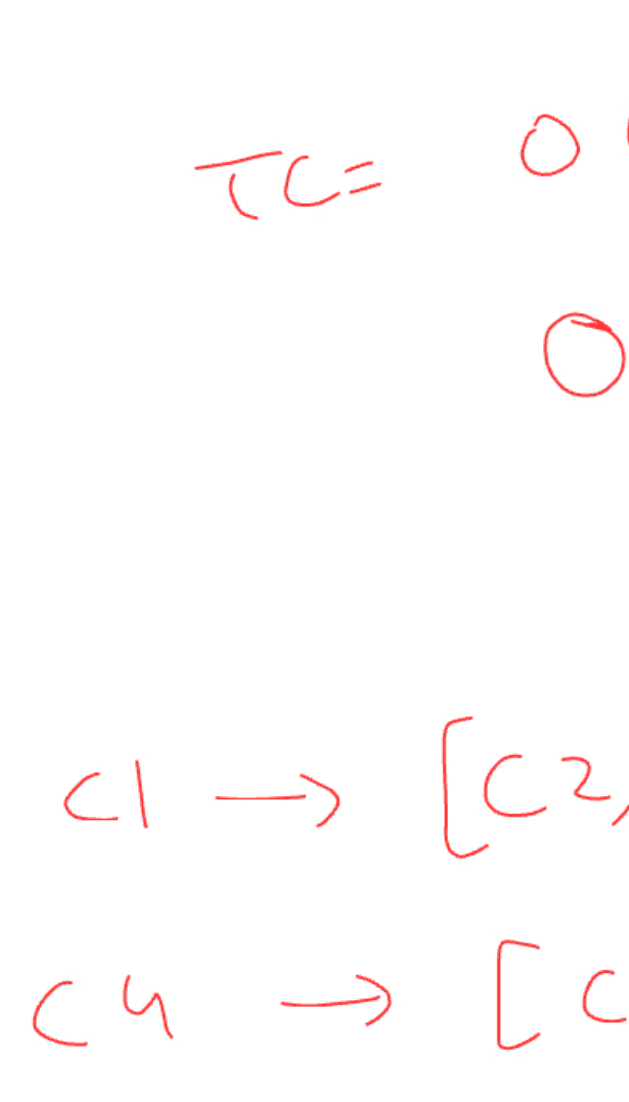
target = 5 k = 3

o/p : [3, 9, 12, 13, ~~8~~, 14]

[12, 13, 9, 14, 3]

pathToTarget = [1, 2, 5]

k=1 k=2 k=3


tar : 5
1
3
10
2
4
9
14
5
6
7
8
11
12
13

currPath : [1, 2, ~~4~~, ~~9~~, ~~14~~, 5]

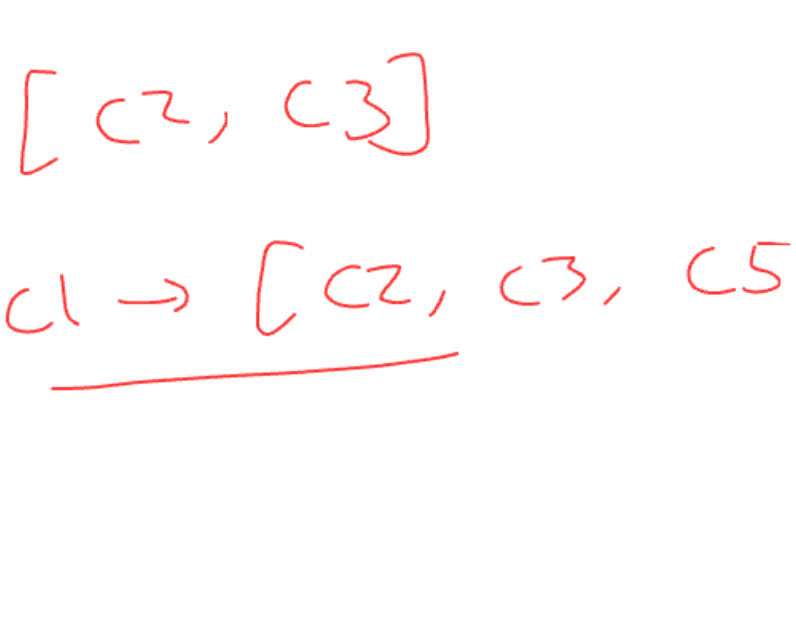
resPath : [1, 2, 5]

TC = O(H * n) ✗

O(n) ✓

U → V

U comes before V


C4
C2
C1
C5
C3

C1 → [C2, C3]

C4 → [C2]

C5 → [C1]

[C2, C3, C4, C1, C5]

if C0 is done → C1 → [C2, C3]

if C0 has not been done → C1 → [C2, C3, C5]

Dynamic Dependency